```
import pandas as pd
import matplotlib.pylab as plt
import numpy as np
import seaborn as sns
```

```
df = pd.read_csv("hotel_bookings.csv")
```

```
df.head(5)
```

	hotel	is_canceled	lead_time	arrival_date_year	arrival_date_month	arrival_date_week_number	arrival_date_day_of_month	st
0	Resort Hotel	0	342	2015	July	27	1	
1	Resort Hotel	0	737	2015	July	27	1	
2	Resort Hotel	0	7	2015	July	27	1	
3	Resort Hotel	0	13	2015	July	27	1	
4	Resort Hotel	0	14	2015	July	27	1	

5 rows × 32 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 119390 entries, 0 to 119389
Data columns (total 32 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   hotel                           119390 non-null  object
 1   is_canceled                     119390 non-null  int64
 2   lead_time                       119390 non-null  int64
 3   arrival_date_year               119390 non-null  int64
 4   arrival_date_month              119390 non-null  object
 5   arrival_date_week_number        119390 non-null  int64
 6   arrival_date_day_of_month       119390 non-null  int64
 7   stays_in_weekend_nights         119390 non-null  int64
 8   stays_in_week_nights            119390 non-null  int64
 9   adults                          119390 non-null  int64
 10  children                        119386 non-null  float64
 11  babies                          119390 non-null  int64
 12  meal                            119390 non-null  object
 13  country                         118902 non-null  object
 14  market_segment                  119390 non-null  object
 15  distribution_channel            119390 non-null  object
 16  is_repeated_guest               119390 non-null  int64
 17  previous_cancellations          119390 non-null  int64
 18  previous_bookings_not_canceled  119390 non-null  int64
 19  reserved_room_type              119390 non-null  object
 20  assigned_room_type              119390 non-null  object
 21  booking_changes                 119390 non-null  int64
 22  deposit_type                    119390 non-null  object
 23  agent                           103050 non-null  float64
 24  company                         6797 non-null    float64
 25  days_in_waiting_list            119390 non-null  int64
 26  customer_type                   119390 non-null  object
 27  adr                             119390 non-null  float64
 28  required_car_parking_spaces     119390 non-null  int64
 29  total_of_special_requests       119390 non-null  int64
 30  reservation_status              119390 non-null  object
 31  reservation_status_date         119390 non-null  object
dtypes: float64(4), int64(16), object(12)
memory usage: 29.1+ MB
```

```
df.describe()
```

	is_canceled	lead_time	arrival_date_year	arrival_date_week_number	arrival_date_day_of_month	stays_in_weekend_nig
count	119390.000000	119390.000000	119390.000000	119390.000000	119390.000000	119390.000
mean	0.370416	104.011416	2016.156554	27.165173	15.798241	0.927
std	0.482918	106.863097	0.707476	13.605138	8.780829	0.998
min	0.000000	0.000000	2015.000000	1.000000	1.000000	0.000
25%	0.000000	18.000000	2016.000000	16.000000	8.000000	0.000
50%	0.000000	69.000000	2016.000000	28.000000	16.000000	1.000
75%	1.000000	160.000000	2017.000000	38.000000	23.000000	2.000
max	1.000000	737.000000	2017.000000	53.000000	31.000000	19.000

```
df.isnull()
```

	hotel	is_canceled	lead_time	arrival_date_year	arrival_date_month	arrival_date_week_number	arrival_date_day_of_mont
0	False	False	False	False	False	False	Fals
1	False	False	False	False	False	False	Fals
2	False	False	False	False	False	False	Fals
3	False	False	False	False	False	False	Fals
4	False	False	False	False	False	False	Fals
...	...	...	...	...	...	...	.
119385	False	False	False	False	False	False	Fals
119386	False	False	False	False	False	False	Fals
119387	False	False	False	False	False	False	Fals
119388	False	False	False	False	False	False	Fals
119389	False	False	False	False	False	False	Fals

119390 rows × 32 columns

```
df.isnull().sum()
```

	0
hotel	0
is_canceled	0
lead_time	0
arrival_date_year	0
arrival_date_month	0
arrival_date_week_number	0
arrival_date_day_of_month	0
stays_in_weekend_nights	0
stays_in_week_nights	0
adults	0
children	4
babies	0
meal	0
country	488
market_segment	0
distribution_channel	0
is_repeated_guest	0
previous_cancellations	0
previous_bookings_not_canceled	0
reserved_room_type	0
assigned_room_type	0
booking_changes	0
deposit_type	0
agent	16340
company	112593
days_in_waiting_list	0
customer_type	0
adr	0
required_car_parking_spaces	0
total_of_special_requests	0
reservation_status	0

```
# Remove rows where the total number of guests is zero
df = df[df['adults'] + df['children'] + df['babies'] > 0]

# Display the shape of the DataFrame after removal to see how many rows were affected
print(f"DataFrame shape after removing invalid bookings: {df.shape}")
```

```
DataFrame shape after removing invalid bookings: (119206, 32)
```

```
df.children[df.children == 0].count()
```

```
np.int64(110616)
```

```
df["children"].unique()
```

```
array([ 0.,  1.,  2., 10.,  3.])
```

```
df["children"] = df["children"].fillna(0)
```

```
df["country"].unique()
```

```
array(['PRT', 'GBR', 'USA', 'ESP', 'IRL', 'FRA', nan, 'ROU', 'NOR', 'OMN',
       'ARG', 'POL', 'DEU', 'BEL', 'CHE', 'CN', 'GRC', 'ITA', 'NLD',
       'DNK', 'RUS', 'SWE', 'AUS', 'EST', 'CZE', 'BRA', 'FIN', 'MOZ',
       'BWA', 'LUX', 'SVN', 'ALB', 'IND', 'CHN', 'MEX', 'MAR', 'UKR',
       'SMR', 'LVA', 'PRI', 'SRB', 'CHL', 'AUT', 'BLR', 'LTU', 'TUR',
       'ZAF', 'AGO', 'ISR', 'CYM', 'ZMB', 'CPV', 'ZWE', 'DZA', 'KOR',
       'CRI', 'HUN', 'ARE', 'TUN', 'JAM', 'HRV', 'HKG', 'IRN', 'GEO',
       'AND', 'GIB', 'URY', 'JEY', 'CAF', 'CYP', 'COL', 'GGY', 'KWT',
       'NGA', 'MDV', 'VEN', 'SVK', 'FJI', 'KAZ', 'PAK', 'IDN', 'LBN',
       'PHL', 'SEN', 'SYC', 'AZE', 'BHR', 'NZL', 'THA', 'DOM', 'MKD',
       'MYS', 'ARM', 'JPN', 'LKA', 'CUB', 'CMR', 'BIH', 'MUS', 'COM',
       'SUR', 'UGA', 'BGR', 'CIV', 'JOR', 'SYR', 'SGP', 'BDI', 'SAU',
       'VNM', 'PLW', 'QAT', 'EGY', 'PER', 'MLT', 'MWI', 'ECU', 'MDG',
       'ISL', 'UZB', 'NPL', 'BHS', 'MAC', 'TGO', 'TWN', 'DJI', 'STP',
       'KNA', 'ETH', 'IRQ', 'HND', 'RWA', 'KHM', 'MCO', 'BGD', 'IMN',
       'TJK', 'NIC', 'BEN', 'VGB', 'TZA', 'GAB', 'GHA', 'TMP', 'GLP',
       'KEN', 'LIE', 'GNB', 'MNE', 'UMI', 'MYT', 'FRO', 'MMR', 'PAN',
       'BFA', 'LBY', 'MLI', 'NAM', 'BOL', 'PRY', 'BRB', 'ABW', 'AIA',
       'SLV', 'DMA', 'PYF', 'GUY', 'LCA', 'ATA', 'GTM', 'ASM', 'MRT',
       'NCL', 'KIR', 'SDN', 'ATF', 'SLE', 'LAO'], dtype=object)
```

```
df["country"] = df["country"].fillna("unknown")
df["agent"] = df["agent"].fillna(df["agent"].mean())
```

```
df["agent"] = df["agent"].fillna(df["agent"].mean())
```

```
df['reservation_status_date'] = pd.to_datetime(df['reservation_status_date'], errors='coerce')
display(df['reservation_status_date'].head())
```

	reservation_status_date
0	2015-07-01
1	2015-07-01
2	2015-07-02
3	2015-07-02
4	2015-07-03

dtype: datetime64[ns]

```
df['reservation_status_date'].info()
```

```
<class 'pandas.core.series.Series'>
Index: 119206 entries, 0 to 119389
Series name: reservation_status_date
Non-Null Count   Dtype
--------------   -----
119206 non-null  datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 5.9 MB
```

Now that reservation_status_date is in datetime format, we can see if there are any NaT values (missing dates) by checking for nulls.

```
df['reservation_status_date'].isnull().sum()
```

```
np.int64(0)
```

```
df.isnull().sum()
```

	0
hotel	0
is_canceled	0
lead_time	0
arrival_date_year	0
arrival_date_month	0
arrival_date_week_number	0
arrival_date_day_of_month	0
stays_in_weekend_nights	0
stays_in_week_nights	0
adults	0
children	0
babies	0
meal	0
country	0
market_segment	0
distribution_channel	0
is_repeated_guest	0
previous_cancellations	0
previous_bookings_not_canceled	0
reserved_room_type	0
assigned_room_type	0
booking_changes	0
deposit_type	0
agent	0
company	0
days_in_waiting_list	0
customer_type	0
adr	0
required_car_parking_spaces	0
total_of_special_requests	0
reservation_status	0

```
df.isnull().sum()
```

total nights	0
revenue	0
arrival_date_day_of_month-binned	0

dtype: int64

	0
hotel	0
is_canceled	0
lead_time	0
arrival_date_year	0
arrival_date_month	0
arrival_date_week_number	0
arrival_date_day_of_month	0
stays_in_weekend_nights	0
stays_in_week_nights	0
adults	0
children	0
babies	0
meal	0
country	0
market_segment	0
distribution_channel	0
is_repeated_guest	0
previous_cancellations	0
previous_bookings_not_canceled	0
reserved_room_type	0
assigned_room_type	0
booking_changes	0
deposit_type	0
agent	0
days_in_waiting_list	0
customer_type	0
adr	0
required_car_parking_spaces	0
total_of_special_requests	0
reservation_status	0
reservation_status_date	0

```
df["total nights"] = df["stays_in_weekend_nights"] + df["stays_in_week_nights"]
df["revenue"] = df["adr"] * df["total nights"]
```

```
df.shape
```

```
(119206, 34)
```

dtype: int64

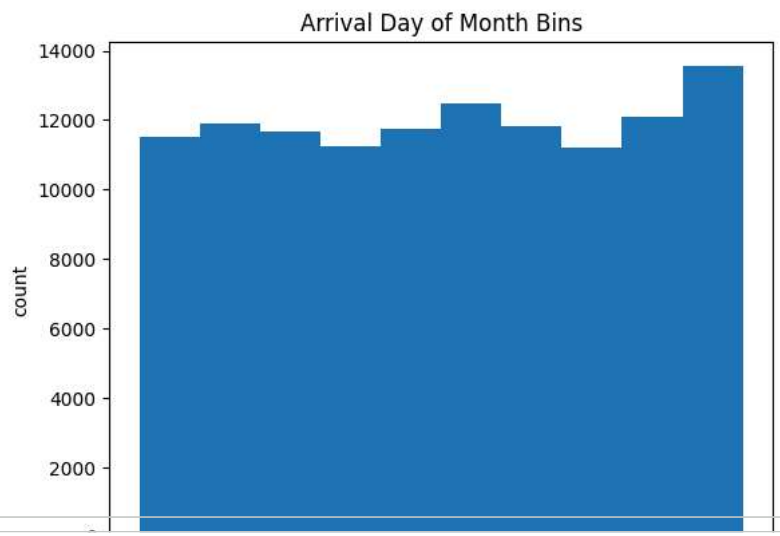
```
df["adr"].describe()
```

	adr
count	119206.000000
mean	101.971519
std	50.432866
min	-6.380000
25%	69.500000
50%	94.950000
75%	126.000000
max	5400.000000

dtype: float64

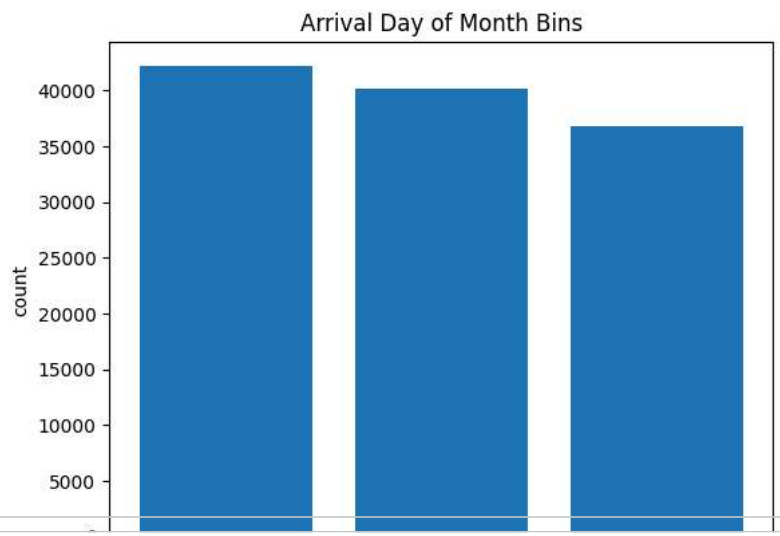
```
%matplotlib inline
import matplotlib as plt
from matplotlib import pyplot

# The column 'arrival_time' does not exist.
# Let's plot the 'arrival_date_day_of_month' instead as an example.
plt.pyplot.hist(df["arrival_date_day_of_month"])

plt.pyplot.xlabel("arrival_date_day_of_month")
plt.pyplot.ylabel("count")
plt.pyplot.title("Arrival Day of Month Bins")
```

```
Text(0.5, 1.0, 'Arrival Day of Month Bins')
```



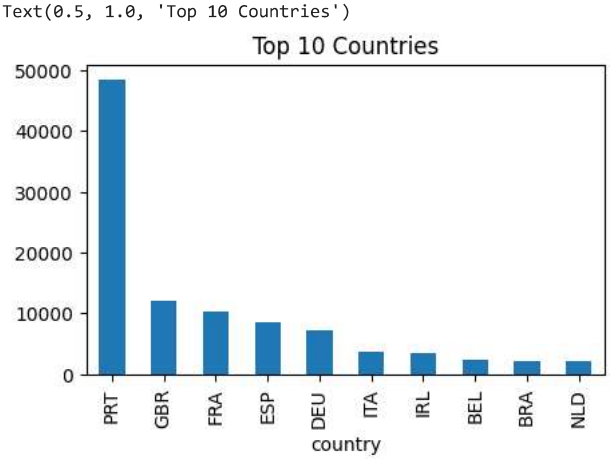
```
%matplotlib inline
import matplotlib as plt
import numpy as np
import pandas as pd
from matplotlib import pyplot

# Define bins for 'arrival_date_day_of_month'
min_day = df["arrival_date_day_of_month"].min()
max_day = df["arrival_date_day_of_month"].max()

# Create 3 bins for the days of the month
bins = np.linspace(min_day, max_day, 4)
group_names = ['Early Month (1-10)', 'Mid Month (11-20)', 'Late Month (21-31)']

# Create the binned column
df['arrival_date_day_of_month-binned'] = pd.cut(df['arrival_date_day_of_month'], bins, labels=group_names, include_lowest=True)

# Plot the bar chart using the binned data
pyplot.bar(group_names, df['arrival_date_day_of_month-binned'].value_counts())

# set x/y labels and plot title
plt.pyplot.xlabel("Arrival Day of Month Bins")
plt.pyplot.ylabel("count")
plt.pyplot.title("Arrival Day of Month Bins")
```

```
Text(0.5, 1.0, 'Arrival Day of Month Bins')
```



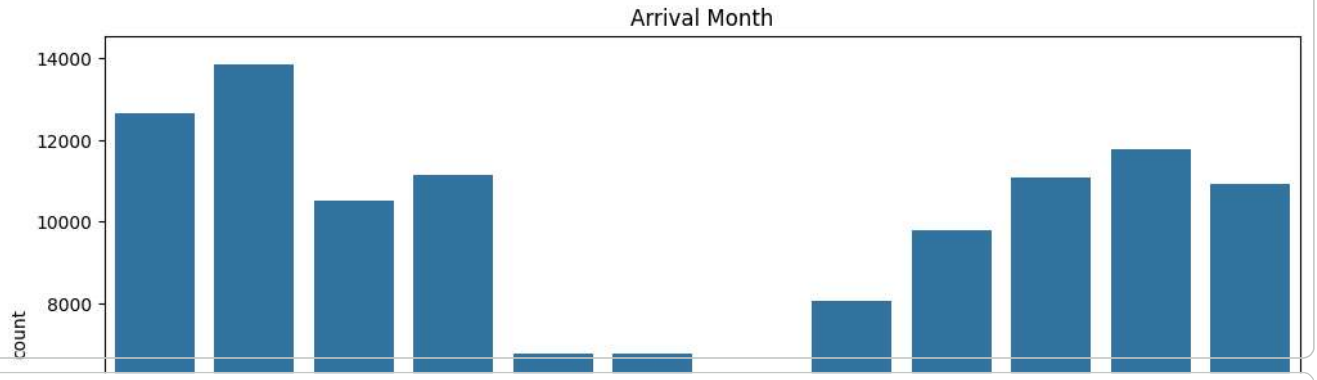

```
pyplot.figure(figsize=(5,3))
df["country"].value_counts()[:10].plot(kind="bar")
pyplot.title("Top 10 Countries")
```

```
Text(0.5, 1.0, 'Top 10 Countries')
```



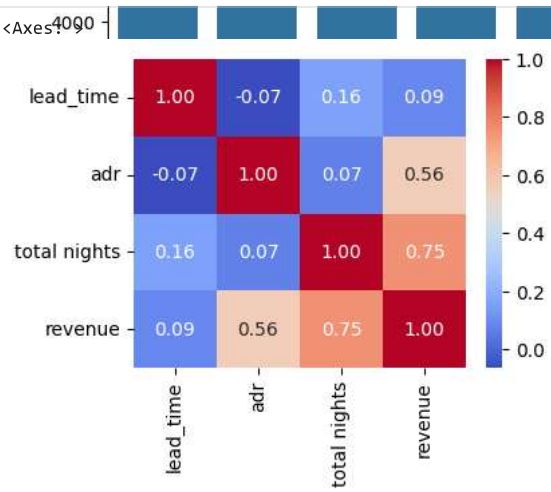

```
pyplot.figure(figsize=(12,6))
sns.countplot(data=df, x="arrival_date_month")
pyplot.title("Arrival Month")
```

```
Text(0.5, 1.0, 'Arrival Month')
```




```
pyplot.figure(figsize=(4,3))
sns.heatmap(corr,annot=True,cmap="coolwarm",fmt=".2f")
```

```
<Axes:>
```



Linear Regression Model to Predict Revenue

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Drop the 'company' column as it was identified as having a high number of nulls and is not suitable for direct imputation or
# We will drop it here explicitly before feature selection for the model.
if 'company' in df.columns:
    df_model = df.drop(columns=['company']).copy()
else:
    df_model = df.copy()

# Define the target variable
y = df_model['revenue']

# Define numerical and categorical features
numerical_features = [
    'lead_time', 'stays_in_weekend_nights', 'stays_in_week_nights',
    'adults', 'children', 'babies', 'previous_cancellations',
    'previous_bookings_not_canceled', 'booking_changes', 'days_in_waiting_list',
    'adr', 'required_car_parking_spaces', 'total_of_special_requests',
    'total nights', 'agent'
]
categorical_features = [
    'hotel', 'arrival_date_month', 'meal', 'country', 'market_segment',
    'distribution_channel', 'deposit_type', 'customer_type', 'reserved_room_type',
    'assigned_room_type', 'arrival_date_day_of_month-binned'
]

# Select only the relevant features for the model
X = df_model[numerical_features + categorical_features]
```

Next, we'll perform one-hot encoding on the categorical features to convert them into a numerical format suitable for linear regression. Due to the large number of unique values in 'country', this might create many new columns. We will proceed with all selected categorical features for now.

```
# Apply one-hot encoding to categorical features
X = pd.get_dummies(X, columns=categorical_features, drop_first=True)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

```
Shape of X_train: (95364, 243)
Shape of X_test: (23842, 243)
Shape of y_train: (95364,)
Shape of y_test: (23842,)
```

Now, we will train a Linear Regression model using the preprocessed data.

```
# Initialize and train the Linear Regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = linear_model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse:.2f}")
print(f"R-squared: {r2:.2f}")

# Display the coefficients of the model
coefficients = pd.DataFrame({'Feature': X_train.columns, 'Coefficient': linear_model.coef_})
display(coefficients.sort_values(by='Coefficient', ascending=False).head(10))
display(coefficients.sort_values(by='Coefficient', ascending=True).head(10))
```

```
Mean Squared Error: 16638.63
R-squared: 0.85
```

	Feature	Coefficient
87	country_FRO	286.354787
240	assigned_room_type_L	267.720167
174	country_SEN	183.482727
229	reserved_room_type_H	130.525785
228	reserved_room_type_G	125.178523
224	reserved_room_type_C	119.624099
34	country_AND	119.055051
90	country_GEO	113.920868
208	market_segment_Complementary	105.991597
194	country_TZA	105.840708

	Feature	Coefficient
74	country_DJI	-518.188077
66	country_COM	-213.236203
113	country_JEY	-196.844839
177	country_SLV	-156.466217
230	reserved_room_type_L	-144.934181
51	country_BHS	-129.111117
37	country_ARM	-128.109378
85	country_FJI	-125.165300
205	country_ZMB	-117.569293
171	country_RWA	-116.776581

```
# Filter coefficients to only include country-specific features
country_coefficients = coefficients[coefficients['Feature'].str.startswith('country_')].copy()

# Extract the country code from the feature name
country_coefficients['Country'] = country_coefficients['Feature'].apply(lambda x: x.split('_')[1])

# Calculate the absolute value of the coefficients
country_coefficients['Absolute_Impact'] = country_coefficients['Coefficient'].abs()

# Sort by absolute impact and display the top 20 countries with the most significant impact
display(country_coefficients.sort_values(by='Absolute_Impact', ascending=False).head(20).drop(columns='Feature'))
```

	Coefficient	Country	Absolute_Impact
74	-518.188077	DJI	518.188077
87	286.354787	FRO	286.354787
66	-213.236203	COM	213.236203
113	-196.844839	JEY	196.844839
174	183.482727	SEN	183.482727
177	-156.466217	SLV	156.466217
51	-129.111117	BHS	129.111117

Quantitative Results Supporting Conclusions

```
print(f"\n--- Linear Regression Model Performance ---")
print(f"Mean Squared Error: {mse:.2f}")
```